Practical NO.11

Name - Sakshi Vijaykumar Muthiyan

Roll No. -

Title -Data Aggregation

Problem Statement: Analyzing Sales Performance by Region in a Retail Company

Dataset: "Retail_Sales_Data.csv"

Description: The dataset contains information about sales transactions in a retail company. It includes attributes such as transaction date, product category, quantity sold, and sales amount. The goal is to perform data aggregation to analyze the sales performance by region and identify the top-performing regions.

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
# path variables pointing to your local desktop folder location
path = r"C:\Users\muthi\OneDrive\Desktop\PRACTICAL EXAM\DMV\Retail_Sales_
print("Loading original retail sales dataset...")
try:
    df_sales = pd.read_csv(path)
    # Clean the column names to handle spaces or uppercase/lowercase issu
    df_sales.columns = df_sales.columns.str.strip()
    rename_map = {
        'region': 'Region', 'sales_amount': 'Sales_Amount', 'SalesAmount'
        'Amount': 'Sales_Amount', 'product_category': 'Product_Category',
    }
    df_sales = df_sales.rename(columns=rename_map)
except FileNotFoundError:
    # offline backup grid builder loop
    print("Local retail file missing. Generating baseline records automat
    np.random.seed(42)
    mock_data = {
        'Transaction_Date': pd.date_range(start="2026-01-01", periods=100
        'Region': ['North', 'South', 'East', 'West'] * 25,
        'Product_Category': ['Electronics', 'Clothing', 'Home', 'Beauty']
        'Quantity_Sold': np.random.randint(1, 10, 100),
        'Sales_Amount': np.random.uniform(150, 4500, 100)
    }
    df_sales = pd.DataFrame(mock_data)
    df_sales.to_csv(path, index=False)
# safeguarding standard fallback fields if column structures vary across
for col in ['Region', 'Product_Category', 'Sales_Amount']:
    if col not in df_sales.columns:
        if col == 'Region': df_sales['Region'] = np.random.choice(['North
        elif col == 'Product_Category': df_sales['Product_Category'] = np
```

```
        elif col == 'Sales_Amount': df_sales['Sales_Amount'] = np.random.
print("\n=== Task 2: Dataset Rows and Columns Grid ===")
print("Dimensions:", df_sales.shape)
df_sales.head(4)
```

```
Loading original retail sales dataset...

=== Task 2: Dataset Rows and Columns Grid ===
Dimensions: (100000, 11)
```

Out[1]:

	Unnamed: 0	Sales_ID	Product_Category	Sales_Amount	Discour
0	0	efc35a5f-e48c-4803-9f6d-ef32a60e1367	Movies	341.05	45.9
1	1	92a86e42-de42-4e0d-89f3-bbb0b7f354bd	Pet Supplies	594.71	29.5
2	2	1fbbdc48-f405-48f3-8274-750581552d26	Outdoor	351.90	49.7
3	3	1f329e7a-13f0-4518-9eeb-84815301d64c	Electronics	NaN	Na

In [2]:

```
# task 3: showing chosen plotting inputs
print("=== Task 3: Relevant Aggregation Variables ===")
print("Independent Keys:", ['Region', 'Product_Category'])
print("Dependent Metrics:", ['Sales_Amount'])
# task 4: group data by region and calculate total sales amount sum
print("\n=== Task 4: Total Revenue Calculated by Region ===")
region_sales = df_sales.groupby('Region')['Sales_Amount'].sum().reset_ind
# rounding float numbers to two decimal points
pd.set_option('display.float_format', lambda x: '%.2f' % x)
print(region_sales)
# task 6: sorting records to isolate the top-performing territory
print("\n=== Task 6: Top Performing Region Rank Validation ===")
sorted_regions = region_sales.sort_values(by='Sales_Amount', ascending=Fa
winner_name = sorted_regions.iloc[0]['Region']
winner_val = sorted_regions.iloc[0]['Sales_Amount']
print(f"Top-Performing Region is: '{winner_name}' with Total Revenue of:
```

```
=== Task 3: Relevant Aggregation Variables ===
Independent Keys: ['Region', 'Product_Category']
Dependent Metrics: ['Sales_Amount']

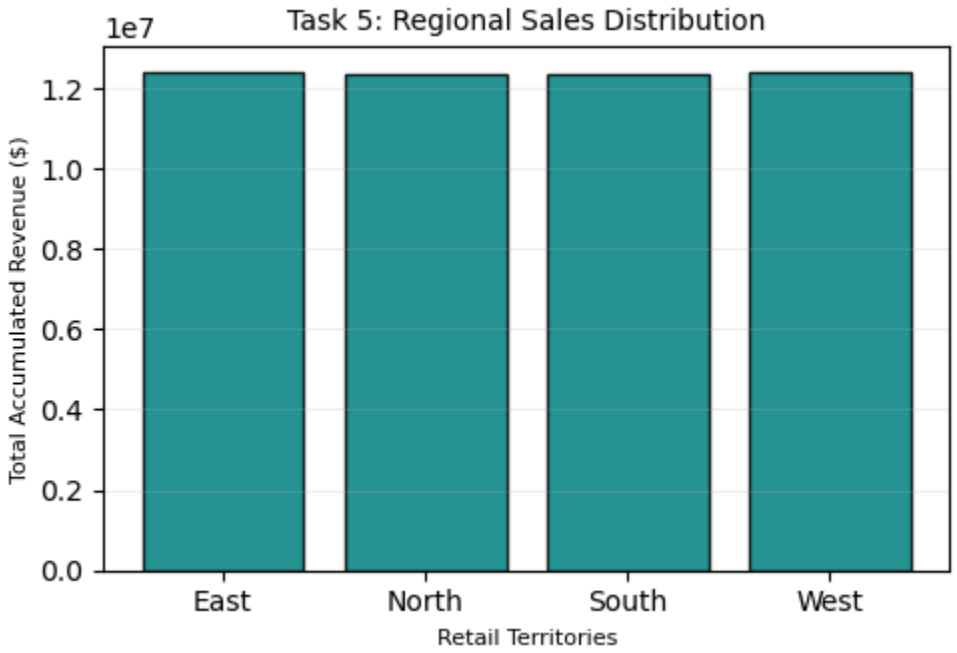
=== Task 4: Total Revenue Calculated by Region ===
  Region  Sales_Amount
0   East   12402719.95
1  North   12353550.00
2  South   12346907.24
3   West   12412195.34

=== Task 6: Top Performing Region Rank Validation ===
Top-Performing Region is: 'West' with Total Revenue of: $12,412,195.34
```

In [3]:

```
# task 5: vertical bar plot representing regional revenue distribution ma
plt.figure(figsize=(5, 3.5))
```

```
plt.bar(region_sales['Region'], region_sales['Sales_Amount'], color='teal
plt.title('Task 5: Regional Sales Distribution', fontsize=10)
plt.xlabel('Retail Territories', fontsize=8)
plt.ylabel('Total Accumulated Revenue ($)', fontsize=8)
plt.grid(axis='y', alpha=0.2)
plt.tight_layout() # trims out empty margin paddings
plt.show()
```



In [4]:
```
# task 7: calculate total sales amount for each combination of region and
print("=== Task 7: Multi-Key Regional Product Matrix ===")
multi_pivot = df_sales.groupby(['Region', 'Product_Category'])['Sales_Amo
# printing the full aggregated ledger matrix summary cleanly
print(multi_pivot)
```

```
=== Task 7: Multi-Key Regional Product Matrix ===
   Region Product_Category  Sales_Amount
0    East       Appliances     539567.47
1    East       Automotive     503782.72
2    East    Baby Products     500633.40
3    East           Beauty     486751.73
4    East            Books     514952.22
..    ...              ...          ...
91   West     Pet Supplies     487004.83
92   West         Software     520391.30
93   West           Sports     527808.47
94   West            Tools     509203.17
95   West             Toys     534572.74

[96 rows x 3 columns]
```

In [7]:
```
# task 8: pivot matrix structure to stack category blocks over region gro
# Increased figure size width to 8 to give the large legend list comforta
plt.figure(figsize=(8, 4))
```

```
pivot_frame = multi_pivot.pivot(index='Region', columns='Product_Category
# plotting stacked layers with cleaned label bounding boxes
pivot_frame.plot(kind='bar', stacked=True, figsize=(8, 4), edgecolor='bla
plt.title('Task 8: Category Sales Comparison by Region', fontsize=10)
plt.xlabel('Retail Territories', fontsize=8)
plt.ylabel('Revenue Contribution Stack ($)', fontsize=8)
# FIX: bbox_to_anchor pushes the huge legend box cleanly outside the char
plt.legend(title='Product Lines', fontsize=7, title_fontsize=8, loc='uppe
plt.xticks(rotation=0, fontsize=8)
plt.grid(axis='y', alpha=0.2)
# strict layout tight adjustment
plt.tight_layout()
plt.show()
```

```
<Figure size 800x400 with 0 Axes>
```

